

File Structure

Every `.ctxdsl` file is wrapped in a single context:

```
context my_system {
  alphabet { ... }
  automata { ... }
  mu_formulas { ... }
}
```

Sections can appear in any order. One context per file.

Alphabet

Declares the set of observable events/signals:

```
alphabet {
  label unlock;
  label lock;
  label bad_code;
}
```

Labels are global to the context. If two automata share a label name, it becomes a *synchronization point* during composition.

Automaton

A finite-state machine lives inside the `automata` block:

```
automata {
  automaton DoorLock {
    states { ... }
    transitions { ... }
  }
}
```

You can define multiple automata in the same context.

States

Declare states inside the automaton. Exactly one must be `initial`:

```
states {
  state Locked initial;
  state Unlocked;
  state Strikel;
  state Lockout;
}
```

State names become predicates you can reference in mu-calculus formulas.

Transitions

Each transition connects two states on a label:

```
transitions {
  transition Locked -> Unlocked
    on label unlock;
  transition Unlocked -> Locked
    on label lock;
  // Self-loop
  transition Locked -> Locked
    on label timeout;
}
```

A state with no outgoing transitions is a *deadlock*.

State Predicates in Formulas

State names double as atomic propositions. You can write:

```
body = mu X. (Lockout || <> X);
```

This checks reachability of the `Lockout` state.

Mu-Calculus Basics

Formulas live in `mu_formulas`. Two fixpoints:

<code>mu X. φ</code>	Greatest fixpoint (safety)
<code>mu X. φ</code>	Least fixpoint (reachability)
<code>$\Box \varphi$</code>	Box — all successors satisfy φ
<code>$\langle \rangle \varphi$</code>	Diamond — some successor satisfies φ

Common patterns:

```
// Safety: always valid
nu X. (Box X)
// Reachability: target reachable
mu X. (Target || <> X)
```

CLI Commands

```
# Evaluate a formula
mununu context eval file.ctxdsl \
  --formula safety_invariant \
  --automaton DoorLock

# Show context summary
mununu context summarize file.ctxdsl
```

First Steps

Always start by checking `safety_invariant` (`nu X. (Box X)`) to confirm your model has no unexpected deadlocks or missing transitions before writing complex properties.